

Code Snippets

```
import sys
print(sys.version_info)
print(sys.version)

>>>
sys.version_info(major=3, minor=4, micro=2,
↳releaselevel='final', serial=0)
3.4.2 (default, Oct 19 2014, 17:52:17)
[GCC 4.2.1 Compatible Apple LLVM 6.0 (clang-600.0.51)]
```

```
def to_str(bytes_or_str):  
    if isinstance(bytes_or_str, bytes):  
        value = bytes_or_str.decode('utf-8')  
    else:  
        value = bytes_or_str  
    return value # Instance of str
```

```
def to_bytes(bytes_or_str):  
    if isinstance(bytes_or_str, str):  
        value = bytes_or_str.encode('utf-8')  
    else:  
        value = bytes_or_str  
    return value    # Instance of bytes
```

Python 2

```
def to_unicode(unicode_or_str):  
    if isinstance(unicode_or_str, str):  
        value = unicode_or_str.decode('utf-8')  
    else:  
        value = unicode_or_str  
    return value # Instance of unicode
```


Python 2

```
def to_str(unicode_or_str):  
    if isinstance(unicode_or_str, unicode):  
        value = unicode_or_str.encode('utf-8')  
    else:  
        value = unicode_or_str  
    return value    # Instance of str
```

```
with open('/tmp/random.bin', 'w') as f:  
    f.write(os.urandom(10))
```

```
>>>
```

```
TypeError: must be str, not bytes
```

```
with open('/tmp/random.bin', 'wb') as f:  
    f.write(os.urandom(10))
```

```
from urllib.parse import parse_qs
my_values = parse_qs('red=5&blue=0&green=',
                      keep_blank_values=True)
print(repr(my_values))
```

```
>>>
```

```
{'red': ['5'], 'green': [''], 'blue': ['0']}
```

```
print('Red:      ', my_values.get('red'))  
print('Green:    ', my_values.get('green'))  
print('Opacity:  ', my_values.get('opacity'))
```

```
>>>
```

```
Red:      ['5']
```

```
Green:    ['']
```

```
Opacity:  None
```

```
# For query string 'red=5&blue=0&green='
red = my_values.get('red', [''])[0] or 0
green = my_values.get('green', [''])[0] or 0
opacity = my_values.get('opacity', [''])[0] or 0
print('Red:      %r' % red)
print('Green:    %r' % green)
print('Opacity: %r' % opacity)
```

```
>>>
Red:      '5'
Green:    0
Opacity: 0
```

```
red = int(my_values.get('red', [''])[0] or 0)
```

```
red = my_values.get('red', [''])  
red = int(red[0]) if red[0] else 0
```



```
green = my_values.get('green', [''])  
if green[0]:  
    green = int(green[0])  
else:  
    green = 0
```

```
def get_first_int(values, key, default=0):  
    found = values.get(key, [''])  
    if found[0]:  
        found = int(found[0])  
    else:  
        found = default  
    return found
```

```
green = get_first_int(my_values, 'green')
```

```
a = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']  
print('First four:', a[:4])  
print('Last four: ', a[-4:])  
print('Middle two:', a[3:-3])
```

```
>>>
```

```
First four: ['a', 'b', 'c', 'd']  
Last four:  ['e', 'f', 'g', 'h']  
Middle two: ['d', 'e']
```

```
a[:]      # ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
a[:5]     # ['a', 'b', 'c', 'd', 'e']
a[:-1]    # ['a', 'b', 'c', 'd', 'e', 'f', 'g']
a[4:]     #           ['e', 'f', 'g', 'h']
a[-3:]    #           ['f', 'g', 'h']
a[2:5]    #           ['c', 'd', 'e']
a[2:-1]   #           ['c', 'd', 'e', 'f', 'g']
a[-3:-1]  #           ['f', 'g']
```

```
a[20]
```

```
>>>
```

```
IndexError: list index out of range
```

```
b = a[4:]  
print('Before: ', b)  
b[1] = 99  
print('After: ', b)  
print('No change:', a)
```

```
>>>  
Before:      ['e', 'f', 'g', 'h']  
After:       ['e', 99, 'g', 'h']  
No change:  ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

```
print('Before ', a)
a[2:7] = [99, 22, 14]
print('After ', a)
>>>
Before  ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
After   ['a', 'b', 99, 22, 14, 'h']
```



```
b = a[:]
```

```
assert b == a and b is not a
```

```
b = a
print('Before', a)
a[:] = [101, 102, 103]
assert a is b          # Still the same list object
print('After ', a)     # Now has different contents

>>>
Before ['a', 'b', 99, 22, 14, 'h']
After  [101, 102, 103]
```

```
a = ['red', 'orange', 'yellow', 'green', 'blue', 'purple']
odds = a[::2]
evens = a[1::2]
print(odds)
print(evens)

>>>
['red', 'yellow', 'blue']
['orange', 'green', 'purple']
```

```
w = '謝謝'  
x = w.encode('utf-8')  
y = x[::-1]  
z = y.decode('utf-8')
```

```
>>>
```

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x9d in  
position 0: invalid start byte
```

```
a = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']  
a[::2]    # ['a', 'c', 'e', 'g']  
a[:: -2]  # ['h', 'f', 'd', 'b']
```

```
a[2::2]      # ['c', 'e', 'g']  
a[-2::-2]    # ['g', 'e', 'c', 'a']  
a[-2:2:-2]   # ['g', 'e']  
a[2:2:-2]    # []
```

```
b = a[::2]    # ['a', 'c', 'e', 'g']  
c = b[1:-1]  # ['c', 'e']
```

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
squares = [x**2 for x in a]  
print(squares)
```

```
>>>
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```



```
squares = map(lambda x: x ** 2, a)
```

```
even_squares = [x**2 for x in a if x % 2 == 0]  
print(even_squares)
```

```
>>>
```

```
[4, 16, 36, 64, 100]
```

```
alt = map(lambda x: x**2, filter(lambda x: x % 2 == 0, a))  
assert even_squares == list(alt)
```

```
chile_ranks = {'ghost': 1, 'habanero': 2, 'cayenne': 3}
rank_dict = {rank: name for name, rank in chile_ranks.items()}
chile_len_set = {len(name) for name in rank_dict.values()}
print(rank_dict)
print(chile_len_set)
```

```
>>>
```

```
{1: 'ghost', 2: 'habanero', 3: 'cayenne'}
{8, 5, 7}
```

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
flat = [x for row in matrix for x in row]  
print(flat)
```

```
>>>
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
squared = [[x**2 for x in row] for row in matrix]  
print(squared)
```

```
>>>
```

```
[[1, 4, 9], [16, 25, 36], [49, 64, 81]]
```

```
my_lists = [  
    [[1, 2, 3], [4, 5, 6]],  
    # ...  
]  
flat = [x for sublist1 in my_lists  
        for sublist2 in sublist1  
        for x in sublist2]
```

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
b = [x for x in a if x > 4 if x % 2 == 0]  
c = [x for x in a if x > 4 and x % 2 == 0]
```



```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
filtered = [[x for x in row if x % 3 == 0]  
            for row in matrix if sum(row) >= 10]  
print(filtered)
```

```
>>>
```

```
[[6], [9]]
```

```
value = [len(x) for x in open('/tmp/my_file.txt')]
print(value)
```

```
>>>
```

```
[100, 57, 15, 1, 12, 75, 5, 86, 89, 11]
```

```
it = (len(x) for x in open('/tmp/my_file.txt'))  
print(it)
```

```
>>>
```

```
<generator object <genexpr> at 0x101b81480>
```

```
roots = ((x, x**0.5) for x in it)
```

```
flavor_list = ['vanilla', 'chocolate', 'pecan', 'strawberry']  
for flavor in flavor_list:  
    print('%s is delicious' % flavor)
```

```
for i in range(len(flavor_list)):
    flavor = flavor_list[i]
    print('%d: %s' % (i + 1, flavor))
```

```
for i, flavor in enumerate(flavor_list):  
    print('%d: %s' % (i + 1, flavor))
```

```
>>>
```

```
1: vanilla
```

```
2: chocolate
```

```
3: pecan
```

```
4: strawberry
```

```
for i, flavor in enumerate(flavor_list, 1):  
    print('%d: %s' % (i, flavor))
```



```
names = ['Cecilia', 'Lise', 'Marie']  
letters = [len(n) for n in names]
```

```
longest_name = None
max_letters = 0

for i in range(len(names)):
    count = letters[i]
    if count > max_letters:
        longest_name = names[i]
        max_letters = count

print(longest_name)

>>>
Cecilia
```

```
for i, name in enumerate(names):  
    count = letters[i]  
    if count > max_letters:  
        longest_name = name  
        max_letters = count
```

```
for name, count in zip(names, letters):  
    if count > max_letters:  
        longest_name = name  
        max_letters = count
```

```
names.append('Rosalind')  
for name, count in zip(names, letters):  
    print(name)
```

```
>>>
```

```
Cecilia
```

```
Lise
```

```
Marie
```

```
for x in []:  
    print('Never runs')  
else:  
    print('For Else block!')
```

```
>>>
```

```
For Else block!
```

```
while False:
    print('Never runs')
else:
    print('While Else block!')

>>>
While Else block!
```

```
a = 4
b = 9
for i in range(2, min(a, b) + 1):
    print('Testing', i)
    if a % i == 0 and b % i == 0:
        print('Not coprime')
        break
else:
    print('Coprime')
```

```
>>>
```

```
Testing 2
```

```
Testing 3
```

```
Testing 4
```

```
Coprime
```



```
def coprime(a, b):  
    for i in range(2, min(a, b) + 1):  
        if a % i == 0 and b % i == 0:  
            return False  
    return True
```

```
def coprime2(a, b):  
    is_coprime = True  
    for i in range(2, min(a, b) + 1):  
        if a % i == 0 and b % i == 0:  
            is_coprime = False  
            break  
    return is_coprime
```

```
handle = open('/tmp/random_data.txt') # May raise IOError
try:
    data = handle.read() # May raise UnicodeDecodeError
finally:
    handle.close() # Always runs after try:
```

```
def load_json_key(data, key):  
    try:  
        result_dict = json.loads(data) # May raise ValueError  
    except ValueError as e:  
        raise KeyError from e  
    else:  
        return result_dict[key] # May raise KeyError
```

```
UNDEFINED = object()
```

```
def divide_json(path):  
    handle = open(path, 'r+') # May raise IOError  
    try:  
        data = handle.read() # May raise UnicodeDecodeError  
        op = json.loads(data) # May raise ValueError  
        value = (  
            op['numerator'] /  
            op['denominator']) # May raise ZeroDivisionError  
    except ZeroDivisionError as e:  
        return UNDEFINED  
    else:  
        op['result'] = value  
        result = json.dumps(op)  
        handle.seek(0)  
        handle.write(result) # May raise IOError  
        return value  
    finally:  
        handle.close() # Always runs
```

```
x, y = 0, 5
result = divide(x, y)
if not result:
    print('Invalid inputs') # This is wrong!
```

```
success, result = divide(x, y)
if not success:
    print('Invalid inputs')
```

```
def divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError as e:  
        raise ValueError('Invalid inputs') from e
```



```
x, y = 5, 2
try:
    result = divide(x, y)
except ValueError:
    print('Invalid inputs')
else:
    print('Result is %.1f' % result)

>>>
Result is 2.5
```

```
def sort_priority(values, group):  
    def helper(x):  
        if x in group:  
            return (0, x)  
        return (1, x)  
    values.sort(key=helper)
```

```
numbers = [8, 3, 1, 2, 5, 4, 7, 6]  
group = {2, 3, 5, 7}  
sort_priority(numbers, group)  
print(numbers)
```

```
>>>
```

```
[2, 3, 5, 7, 1, 4, 6, 8]
```

```
def sort_priority2(numbers, group):  
    found = False  
    def helper(x):  
        if x in group:  
            found = True # Seems simple  
            return (0, x)  
        return (1, x)  
    numbers.sort(key=helper)  
    return found
```

```
found = sort_priority2(numbers, group)
print('Found:', found)
print(numbers)
```

```
>>>
```

```
Found: False
```

```
[2, 3, 5, 7, 1, 4, 6, 8]
```

```
def sort_priority2(numbers, group):  
    found = False          # Scope: 'sort_priority2'  
    def helper(x):  
        if x in group:  
            found = True    # Scope: 'helper' -- Bad!  
            return (0, x)  
        return (1, x)  
    numbers.sort(key=helper)  
    return found
```

```
def sort_priority3(numbers, group):  
    found = False  
    def helper(x):  
        nonlocal found  
        if x in group:  
            found = True  
            return (0, x)  
        return (1, x)  
    numbers.sort(key=helper)  
    return found
```

```
class Sorter(object):  
    def __init__(self, group):  
        self.group = group  
        self.found = False  
  
    def __call__(self, x):  
        if x in self.group:  
            self.found = True  
            return (0, x)  
        return (1, x)  
  
sorter = Sorter(group)  
numbers.sort(key=sorter)  
assert sorter.found is True
```


Python 2

```
def sort_priority(numbers, group):  
    found = [False]  
    def helper(x):  
        if x in group:  
            found[0] = True  
            return (0, x)  
        return (1, x)  
    numbers.sort(key=helper)  
    return found[0]
```

```
def index_words(text):  
    result = []  
    if text:  
        result.append(0)  
    for index, letter in enumerate(text):  
        if letter == ' ':  
            result.append(index + 1)  
    return result
```

```
address = 'Four score and seven years ago...'  
result = index_words(address)  
print(result[:3])
```

```
>>>
```

```
[0, 5, 11]
```

```
def index_words_iter(text):
    if text:
        yield 0
    for index, letter in enumerate(text):
        if letter == ' ':
            yield index + 1
```

```
result = list(index_words_iter(address))
```

```
with open('/tmp/address.txt', 'r') as f:  
    it = index_file(f)  
    results = islice(it, 0, 3)  
    print(list(results))
```

```
>>>
```

```
[0, 5, 11]
```

```
def normalize(numbers):  
    total = sum(numbers)  
    result = []  
    for value in numbers:  
        percent = 100 * value / total  
        result.append(percent)  
    return result
```

```
visits = [15, 35, 80]
percentages = normalize(visits)
print(percentages)

>>>
[11.538461538461538, 26.923076923076923, 61.53846153846154]
```



```
def read_visits(data_path):  
    with open(data_path) as f:  
        for line in f:  
            yield int(line)
```

```
it = read_visits('/tmp/my_numbers.txt')
percentages = normalize(it)
print(percentages)
```

```
>>>
```

```
[]
```

```
it = read_visits('/tmp/my_numbers.txt')  
print(list(it))  
print(list(it))    # Already exhausted
```

```
>>>
```

```
[15, 35, 80]
```

```
[]
```

```
def normalize_copy(numbers):  
    numbers = list(numbers) # Copy the iterator  
    total = sum(numbers)  
    result = []  
    for value in numbers:  
        percent = 100 * value / total  
        result.append(percent)  
    return result
```

```
it = read_visits('/tmp/my_numbers.txt')
percentages = normalize_copy(it)
print(percentages)

>>>
[11.538461538461538, 26.923076923076923, 61.53846153846154]
```

```
def normalize_func(get_iter):  
    total = sum(get_iter())    # New iterator  
    result = []  
    for value in get_iter():    # New iterator  
        percent = 100 * value / total  
        result.append(percent)  
    return result
```

```
percentages = normalize_func(lambda: read_visits(path))
```

```
class ReadVisits(object):  
    def __init__(self, data_path):  
        self.data_path = data_path  
  
    def __iter__(self):  
        with open(self.data_path) as f:  
            for line in f:  
                yield int(line)
```



```
visits = ReadVisits(path)
percentages = normalize(visits)
print(percentages)

>>>
[11.538461538461538, 26.923076923076923, 61.53846153846154]
```

```
def normalize_defensive(numbers):  
    if iter(numbers) is iter(numbers): # An iterator -- bad!  
        raise TypeError('Must supply a container')  
    total = sum(numbers)  
    result = []  
    for value in numbers:  
        percent = 100 * value / total  
        result.append(percent)  
    return result
```

```
visits = [15, 35, 80]
normalize_defensive(visits) # No error
visits = ReadVisits(path)
normalize_defensive(visits) # No error
```

```
it = iter(visits)
normalize_defensive(it)
```

```
>>>
```

```
TypeError: Must supply a container
```

```
def log(message, values):  
    if not values:  
        print(message)  
    else:  
        values_str = ', '.join(str(x) for x in values)  
        print('%s: %s' % (message, values_str))
```

```
log('My numbers are', [1, 2])  
log('Hi there', [])
```

```
>>>
```

```
My numbers are: 1, 2
```

```
Hi there
```

```
def log(message, *values): # The only difference
    if not values:
        print(message)
    else:
        values_str = ', '.join(str(x) for x in values)
        print('%s: %s' % (message, values_str))
```

```
log('My numbers are', 1, 2)
log('Hi there') # Much better
```

```
>>>
```

```
My numbers are: 1, 2
```

```
Hi there
```

```
favorites = [7, 33, 99]  
log('Favorite colors', *favorites)
```

```
>>>
```

```
Favorite colors: 7, 33, 99
```

```
def my_generator():  
    for i in range(10):  
        yield i
```

```
def my_func(*args):  
    print(args)
```

```
it = my_generator()  
my_func(*it)
```

```
>>>  
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```



```
def log(sequence, message, *values):
    if not values:
        print('%s: %s' % (sequence, message))
    else:
        values_str = ', '.join(str(x) for x in values)
        print('%s: %s: %s' % (sequence, message, values_str))
```

```
log(1, 'Favorites', 7, 33)      # New usage is OK
log('Favorite numbers', 7, 33)  # Old usage breaks
```

```
>>>
1: Favorites: 7, 33
Favorite numbers: 7: 33
```

```
def remainder(number, divisor):  
    return number % divisor
```

```
assert remainder(20, 7) == 6
```

remainder(20, 7)

remainder(20, divisor=7)

remainder(number=20, divisor=7)

remainder(divisor=7, number=20)

```
remainder(number=20, 7)
```

```
>>>
```

```
SyntaxError: non-keyword arg after keyword arg
```

```
remainder(20, number=7)
```

```
>>>
```

```
TypeError: remainder() got multiple values for argument  
↳ 'number'
```

```
def flow_rate(weight_diff, time_diff):  
    return weight_diff / time_diff  
  
weight_diff = 0.5  
time_diff = 3  
flow = flow_rate(weight_diff, time_diff)  
print('%.3f kg per second' % flow)  
  
>>>  
0.167 kg per second
```

```
def flow_rate(weight_diff, time_diff, period):  
    return (weight_diff / time_diff) * period
```

```
flow_per_second = flow_rate(weight_diff, time_diff, 1)
```



```
def flow_rate(weight_diff, time_diff, period=1):  
    return (weight_diff / time_diff) * period
```

```
flow_per_second = flow_rate(weight_diff, time_diff)
flow_per_hour = flow_rate(weight_diff, time_diff, period=3600)
```

```
def flow_rate(weight_diff, time_diff,
               period=1, units_per_kg=1):
    return ((weight_diff / units_per_kg) / time_diff) * period
```



```
pounds_per_hour = flow_rate(weight_diff, time_diff, 3600, 2.2)
```

```
def log(message, when=datetime.now()):  
    print('%s: %s' % (when, message))
```

```
log('Hi there!')
```

```
sleep(0.1)
```

```
log('Hi again!')
```

```
>>>
```

```
2014-11-15 21:10:10.371432: Hi there!
```

```
2014-11-15 21:10:10.371432: Hi again!
```

```
def log(message, when=None):
    """Log a message with a timestamp.

    Args:
        message: Message to print.
        when: datetime of when the message occurred.
            Defaults to the present time.
    """
    when = datetime.now() if when is None else when
    print('%s: %s' % (when, message))
```

```
log('Hi there!')  
sleep(0.1)  
log('Hi again!')
```

```
>>>
```

```
2014-11-15 21:10:10.472303: Hi there!  
2014-11-15 21:10:10.573395: Hi again!
```



```
def decode(data, default={}):  
    try:  
        return json.loads(data)  
    except ValueError:  
        return default
```

```
def decode(data, default=None):
    """Load JSON data from a string.

    Args:
        data: JSON data to decode.
        default: Value to return if decoding fails.
            Defaults to an empty dictionary.
    """
    if default is None:
        default = {}
    try:
        return json.loads(data)
    except ValueError:
        return default
```

```
def safe_division(number, divisor, ignore_overflow,
                  ignore_zero_division):
    try:
        return number / divisor
    except OverflowError:
        if ignore_overflow:
            return 0
        else:
            raise
    except ZeroDivisionError:
        if ignore_zero_division:
            return float('inf')
        else:
            raise
```

```
result = safe_division(1, 10**500, True, False)
print(result)
```

```
>>>
```

```
0.0
```

```
result = safe_division(1, 0, False, True)
print(result)
```

```
>>>
```

```
inf
```

```
def safe_division_b(number, divisor,  
                    ignore_overflow=False,  
                    ignore_zero_division=False):  
    # ...
```

```
safe_division_b(1, 10**500, ignore_overflow=True)  
safe_division_b(1, 0, ignore_zero_division=True)
```

```
safe_division_b(1, 10**500, True, False)
```



```
def safe_division_c(number, divisor, *,  
                    ignore_overflow=False,  
                    ignore_zero_division=False):  
    # ...
```

```
safe_division_c(1, 10**500, True, False)
```

```
>>>
```

```
TypeError: safe_division_c() takes 2 positional arguments but  
➡4 were given
```

```
safe_division_c(1, 0, ignore_zero_division=True) # OK
```

```
try:
```

```
    safe_division_c(1, 0)
```

```
except ZeroDivisionError:
```

```
    pass # Expected
```

```
# Python 2
```

```
def print_args(*args, **kwargs):  
    print 'Positional:', args  
    print 'Keyword:    ', kwargs
```

```
print_args(1, 2, foo='bar', stuff='meep')
```

```
>>>
```

```
Positional: (1, 2)
```

```
Keyword:    {'foo': 'bar', 'stuff': 'meep'}
```

```
# Python 2
def safe_division_d(number, divisor, **kwargs):
    ignore_overflow = kwargs.pop('ignore_overflow', False)
    ignore_zero_div = kwargs.pop('ignore_zero_division', False)
    if kwargs:
        raise TypeError('Unexpected **kwargs: %r' % kwargs)
    # ...
```

```
safe_division_d(1, 10)  
safe_division_d(1, 0, ignore_zero_division=True)  
safe_division_d(1, 10**500, ignore_overflow=True)
```

```
safe_division_d(1, 0, False, True)
```

```
>>>
```

```
TypeError: safe_division_d() takes 2 positional arguments but 4  
were given
```

```
safe_division_d(0, 0, unexpected=True)
```

```
>>>
```

```
TypeError: Unexpected **kwargs: {'unexpected': True}
```



```
class SimpleGradebook(object):  
    def __init__(self):  
        self._grades = {}  
  
    def add_student(self, name):  
        self._grades[name] = []  
  
    def report_grade(self, name, score):  
        self._grades[name].append(score)  
  
    def average_grade(self, name):  
        grades = self._grades[name]  
        return sum(grades) / len(grades)
```

```
book = SimpleGradebook()
book.add_student('Isaac Newton')
book.report_grade('Isaac Newton', 90)
# ...
print(book.average_grade('Isaac Newton'))

>>>
90.0
```

```
class BySubjectGradebook(object):  
    def __init__(self):  
        self._grades = {}  
    def add_student(self, name):  
        self._grades[name] = {}
```

```
def report_grade(self, name, subject, grade):  
    by_subject = self._grades[name]  
    grade_list = by_subject.setdefault(subject, [])  
    grade_list.append(grade)  
  
def average_grade(self, name):  
    by_subject = self._grades[name]  
    total, count = 0, 0  
    for grades in by_subject.values():  
        total += sum(grades)  
        count += len(grades)  
    return total / count
```

```
book = BySubjectGradebook()  
book.add_student('Albert Einstein')  
book.report_grade('Albert Einstein', 'Math', 75)  
book.report_grade('Albert Einstein', 'Math', 65)  
book.report_grade('Albert Einstein', 'Gym', 90)  
book.report_grade('Albert Einstein', 'Gym', 95)
```

```
class WeightedGradebook(object):  
    # ...  
    def report_grade(self, name, subject, score, weight):  
        by_subject = self._grades[name]  
        grade_list = by_subject.setdefault(subject, [])  
        grade_list.append((score, weight))
```

```
def average_grade(self, name):  
    by_subject = self._grades[name]  
    score_sum, score_count = 0, 0  
    for subject, scores in by_subject.items():  
        subject_avg, total_weight = 0, 0  
        for score, weight in scores:  
            # ...  
    return score_sum / score_count
```

```
book.report_grade('Albert Einstein', 'Math', 80, 0.10)
```



```
grades = []
grades.append((95, 0.45))
# ...
total = sum(score * weight for score, weight in grades)
total_weight = sum(weight for _, weight in grades)
average_grade = total / total_weight
```

```
grades = []
grades.append((95, 0.45, 'Great job'))
# ...
total = sum(score * weight for score, weight, _ in grades)
total_weight = sum(weight for _, weight, _ in grades)
average_grade = total / total_weight
```

```
import collections
Grade = collections.namedtuple('Grade', ('score', 'weight'))
```

```
class Subject(object):
    def __init__(self):
        self._grades = []

    def report_grade(self, score, weight):
        self._grades.append(Grade(score, weight))

    def average_grade(self):
        total, total_weight = 0, 0
        for grade in self._grades:
            total += grade.score * grade.weight
            total_weight += grade.weight
        return total / total_weight
```

```
class Student(object):
    def __init__(self):
        self._subjects = {}

    def subject(self, name):
        if name not in self._subjects:
            self._subjects[name] = Subject()
        return self._subjects[name]

    def average_grade(self):
        total, count = 0, 0
        for subject in self._subjects.values():
            total += subject.average_grade()
            count += 1
        return total / count
```

```
class Gradebook(object):  
    def __init__(self):  
        self._students = {}  
  
    def student(self, name):  
        if name not in self._students:  
            self._students[name] = Student()  
        return self._students[name]
```

```
book = Gradebook()
albert = book.student('Albert Einstein')
math = albert.subject('Math')
math.report_grade(80, 0.10)
# ...
print(albert.average_grade())

>>>
81.5
```

```
names = ['Socrates', 'Archimedes', 'Plato', 'Aristotle']
names.sort(key=lambda x: len(x))
print(names)

>>>
['Plato', 'Socrates', 'Aristotle', 'Archimedes']
```



```
current = {'green': 12, 'blue': 3}
increments = [
    ('red', 5),
    ('blue', 17),
    ('orange', 9),
]

result = defaultdict(log_missing, current)
print('Before:', dict(result))
for key, amount in increments:
    result[key] += amount
print('After: ', dict(result))

>>>
Before: {'green': 12, 'blue': 3}
Key added
Key added
After:  {'orange': 9, 'green': 12, 'blue': 20, 'red': 5}
```

```
def increment_with_report(current, increments):  
    added_count = 0  
  
    def missing():  
        nonlocal added_count # Stateful closure  
        added_count += 1  
        return 0  
  
    result = defaultdict(missing, current)  
    for key, amount in increments:  
        result[key] += amount  
  
    return result, added_count
```

```
result, count = increment_with_report(current, increments)
assert count == 2
```

```
counter = CountMissing()
result = defaultdict(counter.missing, current) # Method ref

for key, amount in increments:
    result[key] += amount
assert counter.added == 2
```

```
class BetterCountMissing(object):  
    def __init__(self):  
        self.added = 0  
  
    def __call__(self):  
        self.added += 1  
        return 0
```

```
counter = BetterCountMissing()  
counter()  
assert callable(counter)
```

```
counter = BetterCountMissing()
result = defaultdict(counter, current) # Relies on __call__
for key, amount in increments:
    result[key] += amount
assert counter.added == 2
```

```
class InputData(object):  
    def read(self):  
        raise NotImplementedError
```

```
class PathInputData(InputData):  
    def __init__(self, path):  
        super().__init__()  
        self.path = path  
  
    def read(self):  
        return open(self.path).read()
```



```
class Worker(object):  
    def __init__(self, input_data):  
        self.input_data = input_data  
        self.result = None  
  
    def map(self):  
        raise NotImplementedError  
  
    def reduce(self, other):  
        raise NotImplementedError
```

```
class LineCountWorker(Worker):  
    def map(self):  
        data = self.input_data.read()  
        self.result = data.count('\n')  
  
    def reduce(self, other):  
        self.result += other.result
```

```
def generate_inputs(data_dir):  
    for name in os.listdir(data_dir):  
        yield PathInputData(os.path.join(data_dir, name))
```

```
def create_workers(input_list):  
    workers = []  
    for input_data in input_list:  
        workers.append(LineCountWorker(input_data))  
    return workers
```

```
def execute(workers):  
    threads = [Thread(target=w.map) for w in workers]  
    for thread in threads: thread.start()  
    for thread in threads: thread.join()  
  
    first, rest = workers[0], workers[1:]  
    for worker in rest:  
        first.reduce(worker)  
    return first.result
```

```
def mapreduce(data_dir):  
    inputs = generate_inputs(data_dir)  
    workers = create_workers(inputs)  
    return execute(workers)
```

```
from tempfile import TemporaryDirectory

def write_test_files(tmpdir):
    # ...

with TemporaryDirectory() as tmpdir:
    write_test_files(tmpdir)
    result = mapreduce(tmpdir)

print('There are', result, 'lines')

>>>
There are 4360 lines
```

```
class GenericInputData(object):  
    def read(self):  
        raise NotImplementedError  
  
    @classmethod  
    def generate_inputs(cls, config):  
        raise NotImplementedError
```



```
class PathInputData(GenericInputData):
    # ...
    def read(self):
        return open(self.path).read()

    @classmethod
    def generate_inputs(cls, config):
        data_dir = config['data_dir']
        for name in os.listdir(data_dir):
            yield cls(os.path.join(data_dir, name))
```

```
class GenericWorker(object):
    # ...
    def map(self):
        raise NotImplementedError

    def reduce(self, other):
        raise NotImplementedError

    @classmethod
    def create_workers(cls, input_class, config):
        workers = []
        for input_data in input_class.generate_inputs(config):
            workers.append(cls(input_data))
        return workers
```

```
class LineCountWorker(GenericWorker):  
    # ...
```

```
def mapreduce(worker_class, input_class, config):  
    workers = worker_class.create_workers(input_class, config)  
    return execute(workers)
```

```
with TemporaryDirectory() as tmpdir:  
    write_test_files(tmpdir)  
    config = {'data_dir': tmpdir}  
    result = mapreduce(LineCountWorker, PathInputData, config)
```

```
class MyBaseClass(object):  
    def __init__(self, value):  
        self.value = value  
  
class MyChildClass(MyBaseClass):  
    def __init__(self):  
        MyBaseClass.__init__(self, 5)
```

```
class OneWay(MyBaseClass, TimesTwo, PlusFive):  
    def __init__(self, value):  
        MyBaseClass.__init__(self, value)  
        TimesTwo.__init__(self)  
        PlusFive.__init__(self)
```

```
foo = OneWay(5)
print('First ordering is (5 * 2) + 5 =', foo.value)

>>>
First ordering is (5 * 2) + 5 = 15
```



```
class AnotherWay(MyBaseClass, PlusFive, TimesTwo):  
    def __init__(self, value):  
        MyBaseClass.__init__(self, value)  
        TimesTwo.__init__(self)  
        PlusFive.__init__(self)
```

```
bar = AnotherWay(5)
print('Second ordering still is', bar.value)

>>>
Second ordering still is 15
```

```
class TimesFive(MyBaseClass):  
    def __init__(self, value):  
        MyBaseClass.__init__(self, value)  
        self.value *= 5
```

```
class PlusTwo(MyBaseClass):  
    def __init__(self, value):  
        MyBaseClass.__init__(self, value)  
        self.value += 2
```

```
class ThisWay(TimesFive, PlusTwo):
    def __init__(self, value):
        TimesFive.__init__(self, value)
        PlusTwo.__init__(self, value)

foo = ThisWay(5)
print('Should be (5 * 5) + 2 = 27 but is', foo.value)

>>>
Should be (5 * 5) + 2 = 27 but is 7
```

Python 2

```
class TimesFiveCorrect(MyBaseClass):  
    def __init__(self, value):  
        super(TimesFiveCorrect, self).__init__(value)  
        self.value *= 5
```

```
class PlusTwoCorrect(MyBaseClass):  
    def __init__(self, value):  
        super(PlusTwoCorrect, self).__init__(value)  
        self.value += 2
```

Python 2

```
class GoodWay(TimesFiveCorrect, PlusTwoCorrect):  
    def __init__(self, value):  
        super(GoodWay, self).__init__(value)
```

```
foo = GoodWay(5)
```

```
print 'Should be 5 * (5 + 2) = 35 and is', foo.value
```

```
>>>
```

```
Should be 5 * (5 + 2) = 35 and is 35
```

```
from pprint import pprint
pprint(GoodWay.mro())
```

```
>>>
```

```
[<class '__main__.GoodWay'>,
<class '__main__.TimesFiveCorrect'>,
<class '__main__.PlusTwoCorrect'>,
<class '__main__.MyBaseClass'>,
<class 'object'>]
```

```
class Explicit(MyBaseClass):
    def __init__(self, value):
        super().__init__(value * 2)

class Implicit(MyBaseClass):
    def __init__(self, value):
        super().__init__(value * 2)

assert Explicit(10).value == Implicit(10).value
```



```
class ToDictMixin(object):  
    def to_dict(self):  
        return self._traverse_dict(self.__dict__)
```

```
def _traverse_dict(self, instance_dict):
    output = {}
    for key, value in instance_dict.items():
        output[key] = self._traverse(key, value)
    return output

def _traverse(self, key, value):
    if isinstance(value, ToDictMixin):
        return value.to_dict()
    elif isinstance(value, dict):
        return self._traverse_dict(value)
    elif isinstance(value, list):
        return [self._traverse(key, i) for i in value]
    elif hasattr(value, '__dict__'):
        return self._traverse_dict(value.__dict__)
    else:
        return value
```

```
class BinaryTree(ToDictMixin):  
    def __init__(self, value, left=None, right=None):  
        self.value = value  
        self.left = left  
        self.right = right
```

```
tree = BinaryTree(10,  
    left=BinaryTree(7, right=BinaryTree(9)),  
    right=BinaryTree(13, left=BinaryTree(11)))  
print(tree.to_dict())  
  
>>>  
{'left': {'left': None,  
          'right': {'left': None, 'right': None, 'value': 9},  
          'value': 7},  
 'right': {'left': {'left': None, 'right': None, 'value': 11},  
          'right': None,  
          'value': 13},  
 'value': 10}
```

```
class BinaryTreeWithParent(BinaryTree):  
    def __init__(self, value, left=None,  
                  right=None, parent=None):  
        super().__init__(value, left=left, right=right)  
        self.parent = parent
```

```
def _traverse(self, key, value):  
    if (isinstance(value, BinaryTreeWithParent) and  
        key == 'parent'):  
        return value.value # Prevent cycles  
    else:  
        return super()._traverse(key, value)
```

```
root = BinaryTreeWithParent(10)
root.left = BinaryTreeWithParent(7, parent=root)
root.left.right = BinaryTreeWithParent(9, parent=root.left)
print(root.to_dict())
```

```
>>>
{'left': {'left': None,
          'parent': 10,
          'right': {'left': None,
                    'parent': 7,
                    'right': None,
                    'value': 9},
          'value': 7},
 'parent': None,
 'right': None,
 'value': 10}
```

```
class NamedSubTree(ToDictMixin):
    def __init__(self, name, tree_with_parent):
        self.name = name
        self.tree_with_parent = tree_with_parent

my_tree = NamedSubTree('foobar', root.left.right)
print(my_tree.to_dict()) # No infinite loop

>>>
{'name': 'foobar',
 'tree_with_parent': {'left': None,
                       'parent': 7,
                       'right': None,
                       'value': 9}}
```



```
class JsonMixin(object):
    @classmethod
    def from_json(cls, data):
        kwargs = json.loads(data)
        return cls(**kwargs)

    def to_json(self):
        return json.dumps(self.to_dict())
```

```
class DatacenterRack(ToDictMixin, JsonMixin):
    def __init__(self, switch=None, machines=None):
        self.switch = Switch(**switch)
        self.machines = [
            Machine(**kwargs) for kwargs in machines]
```

```
class Switch(ToDictMixin, JsonMixin):
    # ...
```

```
class Machine(ToDictMixin, JsonMixin):
    # ...
```

```
serialized = """{
    "switch": {"ports": 5, "speed": 1e9},
    "machines": [
        {"cores": 8, "ram": 32e9, "disk": 5e12},
        {"cores": 4, "ram": 16e9, "disk": 1e12},
        {"cores": 2, "ram": 4e9, "disk": 500e9}
    ]
}"""
```

```
deserialized = DatacenterRack.from_json(serialized)
roundtrip = deserialized.to_json()
assert json.loads(serialized) == json.loads(roundtrip)
```

```
class MyObject(object):  
    def __init__(self):  
        self.public_field = 5  
        self.__private_field = 10  
  
    def get_private_field(self):  
        return self.__private_field
```

```
foo = MyObject()  
assert foo.public_field == 5
```

```
assert foo.get_private_field() == 10
```

```
foo.__private_field
```

```
>>>
```

```
AttributeError: 'MyObject' object has no attribute  
↳ '__private_field'
```

```
class MyOtherObject(object):
    def __init__(self):
        self.__private_field = 71

    @classmethod
    def get_private_field_of_instance(cls, instance):
        return instance.__private_field

bar = MyOtherObject()
assert MyOtherObject.get_private_field_of_instance(bar) == 71
```



```
class MyParentObject(object):
    def __init__(self):
        self.__private_field = 71

class MyChildObject(MyParentObject):
    def get_private_field(self):
        return self.__private_field

baz = MyChildObject()
baz.get_private_field()

>>>
AttributeError: 'MyChildObject' object has no attribute
↳ '_MyChildObject__private_field'
```

```
assert baz._MyParentObject__private_field == 71
```

```
print(baz.__dict__)
```

```
>>>
```

```
{'_MyParentObject__private_field': 71}
```

```
class MyClass(object):  
    def __init__(self, value):  
        self.__value = value  
  
    def get_value(self):  
        return str(self.__value)  
  
foo = MyClass(5)  
assert foo.get_value() == '5'
```

```
class MyIntegerSubclass(MyClass):  
    def get_value(self):  
        return int(self._MyClass__value)  
  
foo = MyIntegerSubclass(5)  
assert foo.get_value() == 5
```

```
class MyBaseClass(object):  
    def __init__(self, value):  
        self.__value = value  
    # ...  
  
class MyClass(MyBaseClass):  
    # ...  
  
class MyIntegerSubclass(MyClass):  
    def get_value(self):  
        return int(self._MyClass__value)
```

```
foo = MyIntegerSubclass(5)
foo.get_value()
```

```
>>>
```

```
AttributeError: 'MyIntegerSubclass' object has no attribute
↳ '_MyClass__value'
```

```
class MyClass(object):  
    def __init__(self, value):  
        # This stores the user-supplied value for the object.  
        # It should be coercible to a string. Once assigned for  
        # the object it should be treated as immutable.  
        self._value = value
```



```
class ApiClass(object):
    def __init__(self):
        self._value = 5

    def get(self):
        return self._value

class Child(ApiClass):
    def __init__(self):
        super().__init__()
        self._value = 'hello' # Conflicts

a = Child()
print(a.get(), 'and', a._value, 'should be different')

>>>
hello and hello should be different
```

```
class ApiClass(object):
    def __init__(self):
        self.__value = 5

    def get(self):
        return self.__value

class Child(ApiClass):
    def __init__(self):
        super().__init__()
        self._value = 'hello' # OK!

a = Child()
print(a.get(), 'and', a._value, 'are different')

>>>
5 and hello are different
```

```
class FrequencyList(list):  
    def __init__(self, members):  
        super().__init__(members)  
  
    def frequency(self):  
        counts = {}  
        for item in self:  
            counts.setdefault(item, 0)  
            counts[item] += 1  
        return counts
```

```
foo = FrequencyList(['a', 'b', 'a', 'c', 'b', 'a', 'd'])
print('Length is', len(foo))
foo.pop()
print('After pop:', repr(foo))
print('Frequency:', foo.frequency())
```

```
>>>
```

```
Length is 7
```

```
After pop: ['a', 'b', 'a', 'c', 'b', 'a']
```

```
Frequency: {'a': 3, 'c': 1, 'b': 2}
```

```
class BinaryNode(object):  
    def __init__(self, value, left=None, right=None):  
        self.value = value  
        self.left = left  
        self.right = right
```

```
class IndexableNode(BinaryNode):
    def _search(self, count, index):
        # ...
        # Returns (found, count)

    def __getitem__(self, index):
        found, _ = self._search(0, index)
        if not found:
            raise IndexError('Index out of range')
        return found.value
```

```
tree = IndexableNode(  
    10,  
    left=IndexableNode(  
        5,  
        left=IndexableNode(2),  
        right=IndexableNode(  
            6, right=IndexableNode(7))),  
    right=IndexableNode(  
        15, left=IndexableNode(11)))
```

```
print('LRR =', tree.left.right.right.value)
print('Index 0 =', tree[0])
print('Index 1 =', tree[1])
print('11 in the tree?', 11 in tree)
print('17 in the tree?', 17 in tree)
print('Tree is', list(tree))
```

```
>>>
```

```
LRR = 7
```

```
Index 0 = 2
```

```
Index 1 = 5
```

```
11 in the tree? True
```

```
17 in the tree? False
```

```
Tree is [2, 5, 6, 7, 10, 11, 15]
```



```
len(tree)
```

```
>>>
```

```
TypeError: object of type 'IndexableNode' has no len()
```

```
class SequenceNode(IndexableNode):
    def __len__(self):
        _, count = self._search(0, None)
        return count

tree = SequenceNode(
    # ...
)

print('Tree has %d nodes' % len(tree))

>>>
Tree has 7 nodes
```

```
from collections.abc import Sequence
```

```
class BadType(Sequence):  
    pass
```

```
foo = BadType()
```

```
>>>
```

```
TypeError: Can't instantiate abstract class BadType with  
➡abstract methods __getitem__, __len__
```

```
class BetterNode(SequenceNode, Sequence):  
    pass
```

```
tree = BetterNode(  
    # ...  
)
```

```
print('Index of 7 is', tree.index(7))  
print('Count of 10 is', tree.count(10))
```

```
>>>
```

```
Index of 7 is 3
```

```
Count of 10 is 1
```

```
r0 = OldResistor(50e3)
print('Before: %5r' % r0.get_ohms())
r0.set_ohms(10e3)
print('After:  %5r' % r0.get_ohms())

>>>
Before: 50000.0
After:  10000.0
```

```
r0.set_ohms(r0.get_ohms() + 5e3)
```

```
class Resistor(object):  
    def __init__(self, ohms):  
        self.ohms = ohms  
        self.voltage = 0  
        self.current = 0
```

```
r1 = Resistor(50e3)  
r1.ohms = 10e3
```

```
class VoltageResistance(Resistor):  
    def __init__(self, ohms):  
        super().__init__(ohms)  
        self._voltage = 0  
  
    @property  
    def voltage(self):  
        return self._voltage  
  
    @voltage.setter  
    def voltage(self, voltage):  
        self._voltage = voltage  
        self.current = self._voltage / self.ohms
```



```
r2 = VoltageResistance(1e3)
print('Before: %5r amps' % r2.current)
r2.voltage = 10
print('After:  %5r amps' % r2.current)
```

```
>>>
```

```
Before:      0 amps
```

```
After:    0.01 amps
```

```
class BoundedResistance(Resistor):
    def __init__(self, ohms):
        super().__init__(ohms)

    @property
    def ohms(self):
        return self._ohms

    @ohms.setter
    def ohms(self, ohms):
        if ohms <= 0:
            raise ValueError('%f ohms must be > 0' % ohms)
        self._ohms = ohms
```

```
r3 = BoundedResistance(1e3)
```

```
r3.ohms = 0
```

```
>>>
```

```
ValueError: 0.000000 ohms must be > 0
```

```
BoundedResistance(-5)
```

```
>>>
```

```
ValueError: -5.000000 ohms must be > 0
```

```
class FixedResistance(Resistor):
    # ...
    @property
    def ohms(self):
        return self._ohms

    @ohms.setter
    def ohms(self, ohms):
        if hasattr(self, '_ohms'):
            raise AttributeError("Can't set attribute")
        self._ohms = ohms
```

```
r4 = FixedResistance(1e3)
```

```
r4.ohms = 2e3
```

```
>>>
```

```
AttributeError: Can't set attribute
```

```
class MysteriousResistor(Resistor):  
    @property  
    def ohms(self):  
        self.voltage = self._ohms * self.current  
        return self._ohms  
    # ...
```

```
r7 = MysteriousResistor(10)
r7.current = 0.01
print('Before: %5r' % r7.voltage)
r7.ohms
print('After:  %5r' % r7.voltage)
```

```
>>>
```

```
Before:      0
```

```
After:      0.1
```



```
class Bucket(object):
    def __init__(self, period):
        self.period_delta = timedelta(seconds=period)
        self.reset_time = datetime.now()
        self.quota = 0

    def __repr__(self):
        return 'Bucket(quota=%d)' % self.quota
```

```
def fill(bucket, amount):  
    now = datetime.now()  
    if now - bucket.reset_time > bucket.period_delta:  
        bucket.quota = 0  
        bucket.reset_time = now  
    bucket.quota += amount
```

```
def deduct(bucket, amount):  
    now = datetime.now()  
    if now - bucket.reset_time > bucket.period_delta:  
        return False  
    if bucket.quota - amount < 0:  
        return False  
    bucket.quota -= amount  
    return True
```

```
if deduct(bucket, 99):  
    print('Had 99 quota')  
else:  
    print('Not enough for 99 quota')  
print(bucket)
```

```
>>>
```

```
Had 99 quota
```

```
Bucket(quota=1)
```

```
if deduct(bucket, 3):  
    print('Had 3 quota')  
else:  
    print('Not enough for 3 quota')  
print(bucket)
```

```
>>>
```

```
Not enough for 3 quota  
Bucket(quota=1)
```

```
class Bucket(object):
    def __init__(self, period):
        self.period_delta = timedelta(seconds=period)
        self.reset_time = datetime.now()
        self.max_quota = 0
        self.quota_consumed = 0

    def __repr__(self):
        return ('Bucket(max_quota=%d, quota_consumed=%d)' %
                (self.max_quota, self.quota_consumed))
```

```
@property  
def quota(self):  
    return self.max_quota - self.quota_consumed
```

```
@quota.setter
def quota(self, amount):
    delta = self.max_quota - amount
    if amount == 0:
        # Quota being reset for a new period
        self.quota_consumed = 0
        self.max_quota = 0
    elif delta < 0:
        # Quota being filled for the new period
        assert self.quota_consumed == 0
        self.max_quota = amount
    else:
        # Quota being consumed during the period
        assert self.max_quota >= self.quota_consumed
        self.quota_consumed += delta
```



```
bucket = Bucket(60)
print('Initial', bucket)
fill(bucket, 100)
print('Filled', bucket)

if deduct(bucket, 99):
    print('Had 99 quota')
else:
    print('Not enough for 99 quota')

print('Now', bucket)

if deduct(bucket, 3):
    print('Had 3 quota')
else:
    print('Not enough for 3 quota')

print('Still', bucket)

>>>
Initial Bucket(max_quota=0, quota_consumed=0)
Filled Bucket(max_quota=100, quota_consumed=0)
Had 99 quota
Now Bucket(max_quota=100, quota_consumed=99)
Not enough for 3 quota
Still Bucket(max_quota=100, quota_consumed=99)
```

```
class Homework(object):
    def __init__(self):
        self._grade = 0

    @property
    def grade(self):
        return self._grade

    @grade.setter
    def grade(self, value):
        if not (0 <= value <= 100):
            raise ValueError('Grade must be between 0 and 100')
        self._grade = value
```

```
class Exam(object):
    def __init__(self):
        self._writing_grade = 0
        self._math_grade = 0

    @staticmethod
    def _check_grade(value):
        if not (0 <= value <= 100):
            raise ValueError('Grade must be between 0 and 100')
```

```
@property
```

```
def writing_grade(self):  
    return self._writing_grade
```

```
@writing_grade.setter
```

```
def writing_grade(self, value):  
    self._check_grade(value)  
    self._writing_grade = value
```

```
@property
```

```
def math_grade(self):  
    return self._math_grade
```

```
@math_grade.setter
```

```
def math_grade(self, value):  
    self._check_grade(value)  
    self._math_grade = value
```

```
class Grade(object):  
    def __get__(*args, **kwargs):  
        # ...  
  
    def __set__(*args, **kwargs):  
        # ...
```

```
class Exam(object):  
    # Class attributes  
    math_grade = Grade()  
    writing_grade = Grade()  
    science_grade = Grade()
```

```
Exam.__dict__['writing_grade'].__set__(exam, 40)
```

```
print(Exam.__dict__['writing_grade'].__get__(exam, Exam))
```

```
class Grade(object):  
    def __init__(self):  
        self._value = 0  
  
    def __get__(self, instance, instance_type):  
        return self._value  
  
    def __set__(self, instance, value):  
        if not (0 <= value <= 100):  
            raise ValueError('Grade must be between 0 and 100')  
        self._value = value
```



```
first_exam = Exam()
first_exam.writing_grade = 82
first_exam.science_grade = 99
print('Writing', first_exam.writing_grade)
print('Science', first_exam.science_grade)

>>>
Writing 82
Science 99
```

```
second_exam = Exam()
second_exam.writing_grade = 75
print('Second', second_exam.writing_grade, 'is right')
print('First ', first_exam.writing_grade, 'is wrong')

>>>
Second 75 is right
First 75 is wrong
```

```
class Grade(object):
    def __init__(self):
        self._values = {}

    def __get__(self, instance, instance_type):
        if instance is None: return self
        return self._values.get(instance, 0)

    def __set__(self, instance, value):
        if not (0 <= value <= 100):
            raise ValueError('Grade must be between 0 and 100')
        self._values[instance] = value
```

```
class Grade(object):  
    def __init__(self):  
        self._values = WeakKeyDictionary()  
    # ...
```

```
class Exam(object):
    math_grade = Grade()
    writing_grade = Grade()
    science_grade = Grade()
first_exam = Exam()
first_exam.writing_grade = 82
second_exam = Exam()
second_exam.writing_grade = 75
print('First ', first_exam.writing_grade, 'is right')
print('Second', second_exam.writing_grade, 'is right')

>>>
First 82 is right
Second 75 is right
```

```
class LazyDB(object):  
    def __init__(self):  
        self.exists = 5  
  
    def __getattr__(self, name):  
        value = 'Value for %s' % name  
        setattr(self, name, value)  
        return value
```

```
data = LazyDB()
print('Before:', data.__dict__)
print('foo:    ', data.foo)
print('After:  ', data.__dict__)

>>>
Before: {'exists': 5}
foo:    Value for foo
After:  {'exists': 5, 'foo': 'Value for foo'}
```

```
class LoggingLazyDB(LazyDB):  
    def __getattr__(self, name):  
        print('Called __getattr__(%s)' % name)  
        return super().__getattr__(name)
```

```
data = LoggingLazyDB()  
print('exists:', data.exists)  
print('foo:    ', data.foo)  
print('foo:    ', data.foo)
```

```
>>>  
exists: 5  
Called __getattr__(foo)  
foo:    Value for foo  
foo:    Value for foo
```



```
class ValidatingDB(object):
    def __init__(self):
        self.exists = 5

    def __getattr__(self, name):
        print('Called __getattr__((%s)' % name)
        try:
            return super().__getattr__(name)
        except AttributeError:
            value = 'Value for %s' % name
            setattr(self, name, value)
            return value
```

```
data = ValidatingDB()
print('exists:', data.exists)
print('foo: ', data.foo)
print('foo: ', data.foo)
```

```
>>>
Called __getattr__((exists)
exists: 5
Called __getattr__((foo)
foo:  Value for foo
Called __getattr__((foo)
foo:  Value for foo
```

```
class MissingPropertyDB(object):  
    def __getattr__(self, name):  
        if name == 'bad_name':  
            raise AttributeError('%s is missing' % name)  
        # ...
```

```
data = MissingPropertyDB()  
data.bad_name
```

```
>>>
```

```
AttributeError: bad_name is missing
```

```
data = LoggingLazyDB()
print('Before:      ', data.__dict__)
print('foo exists: ', hasattr(data, 'foo'))
print('After:      ', data.__dict__)
print('foo exists: ', hasattr(data, 'foo'))

>>>
Before:      {'exists': 5}
Called __getattr__(foo)
foo exists:  True
After:      {'exists': 5, 'foo': 'Value for foo'}
foo exists:  True
```

```
data = ValidatingDB()
print('foo exists: ', hasattr(data, 'foo'))
print('foo exists: ', hasattr(data, 'foo'))

>>>
Called __getattr__(foo)
foo exists:  True
Called __getattr__(foo)
foo exists:  True
```

```
class SavingDB(object):  
    def __setattr__(self, name, value):  
        # Save some data to the DB log  
        # ...  
        super().__setattr__(name, value)
```

```
class LoggingSavingDB(SavingDB):
    def __setattr__(self, name, value):
        print('Called __setattr__(%s, %r)' % (name, value))
        super().__setattr__(name, value)
```

```
data = LoggingSavingDB()
print('Before: ', data.__dict__)
data.foo = 5
print('After: ', data.__dict__)
data.foo = 7
print('Finally:', data.__dict__)
```

```
>>>
Before: {}
Called __setattr__(foo, 5)
After: {'foo': 5}
Called __setattr__(foo, 7)
Finally: {'foo': 7}
```

```
class BrokenDictionaryDB(object):  
    def __init__(self, data):  
        self._data = {}  
  
    def __getattr__(self, name):  
        print('Called __getattr__(%s)' % name)  
        return self._data[name]
```

```
data = BrokenDictionaryDB({'foo': 3})
```

```
data.foo
```

```
>>>
```

```
Called __getattrute__(foo)
```

```
Called __getattrute__(_data)
```

```
Called __getattrute__(_data)
```

```
...
```

```
Traceback ...
```

```
RuntimeError: maximum recursion depth exceeded
```



```
class DictionaryDB(object):  
    def __init__(self, data):  
        self._data = data  
  
    def __getattr__(self, name):  
        data_dict = super().__getattr__('_data')  
        return data_dict[name]
```

```
class Meta(type):
    def __new__(meta, name, bases, class_dict):
        print((meta, name, bases, class_dict))
        return type.__new__(meta, name, bases, class_dict)

class MyClass(object, metaclass=Meta):
    stuff = 123

    def foo(self):
        pass
```

```
>>>
(<class '__main__.Meta'>,
 'MyClass',
 (<class 'object'>,),
 {'__module__': '__main__',
  '__qualname__': 'MyClass',
  'foo': <function MyClass.foo at 0x102c7dd08>,
  'stuff': 123})
```

```
# Python 2
class Meta(type):
    def __new__(meta, name, bases, class_dict):
        # ...

class MyClassInPython2(object):
    __metaclass__ = Meta
    # ...
```

```
class ValidatePolygon(type):
    def __new__(meta, name, bases, class_dict):
        # Don't validate the abstract Polygon class
        if bases != (object,):
            if class_dict['sides'] < 3:
                raise ValueError('Polygons need 3+ sides')
        return type.__new__(meta, name, bases, class_dict)

class Polygon(object, metaclass=ValidatePolygon):
    sides = None # Specified by subclasses

    @classmethod
    def interior_angles(cls):
        return (cls.sides - 2) * 180

class Triangle(Polygon):
    sides = 3
```

```
print('Before class')
class Line(Polygon):
    print('Before sides')
    sides = 1
    print('After sides')
print('After class')
```

```
>>>
```

```
Before class
```

```
Before sides
```

```
After sides
```

```
Traceback ...
```

```
ValueError: Polygons need 3+ sides
```

```
class Serializable(object):  
    def __init__(self, *args):  
        self.args = args  
  
    def serialize(self):  
        return json.dumps({'args': self.args})
```

```
class Point2D(Serializable):
    def __init__(self, x, y):
        super().__init__(x, y)
        self.x = x
        self.y = y

    def __repr__(self):
        return 'Point2D(%d, %d)' % (self.x, self.y)

point = Point2D(5, 3)
print('Object:      ', point)
print('Serialized:', point.serialize())

>>>
Object:      Point2D(5, 3)
Serialized: {"args": [5, 3]}
```



```
class Deserializable(Serializable):  
    @classmethod  
    def deserialize(cls, json_data):  
        params = json.loads(json_data)  
        return cls(*params['args'])
```

```
class BetterPoint2D(Deserializable):
    # ...

point = BetterPoint2D(5, 3)
print('Before:      ', point)
data = point.serialize()
print('Serialized:', data)
after = BetterPoint2D.deserialize(data)
print('After:       ', after)

>>>
Before:      BetterPoint2D(5, 3)
Serialized: {"args": [5, 3]}
After:       BetterPoint2D(5, 3)
```

```
class BetterSerializable(object):
    def __init__(self, *args):
        self.args = args

    def serialize(self):
        return json.dumps({
            'class': self.__class__.__name__,
            'args': self.args,
        })

    def __repr__(self):
        # ...
```

```
registry = {}
```

```
def register_class(target_class):  
    registry[target_class.__name__] = target_class
```

```
def deserialize(data):  
    params = json.loads(data)  
    name = params['class']  
    target_class = registry[name]  
    return target_class(*params['args'])
```

```
class EvenBetterPoint2D(BetterSerializable):
    def __init__(self, x, y):
        super().__init__(x, y)
        self.x = x
        self.y = y

register_class(EvenBetterPoint2D)
```

```
point = EvenBetterPoint2D(5, 3)
print('Before:      ', point)
data = point.serialize()
print('Serialized:', data)
after = deserialize(data)
print('After:       ', after)

>>>
Before:      EvenBetterPoint2D(5, 3)
Serialized: {"class": "EvenBetterPoint2D", "args": [5, 3]}
After:       EvenBetterPoint2D(5, 3)
```

```
class Point3D(BetterSerializable):  
    def __init__(self, x, y, z):  
        super().__init__(x, y, z)  
        self.x = x  
        self.y = y  
        self.z = z
```

Forgot to call register_class! Whoops!

```
class Meta(type):
    def __new__(meta, name, bases, class_dict):
        cls = type.__new__(meta, name, bases, class_dict)
        register_class(cls)
        return cls

class RegisteredSerializable(BetterSerializable,
                             metaclass=Meta):
    pass
```



```
class Vector3D(RegisteredSerializable):
    def __init__(self, x, y, z):
        super().__init__(x, y, z)
        self.x, self.y, self.z = x, y, z
```

```
v3 = Vector3D(10, -7, 3)
print('Before:      ', v3)
data = v3.serialize()
print('Serialized:', data)
print('After:       ', deserialize(data))
```

```
>>>
```

```
Before:      Vector3D(10, -7, 3)
```

```
Serialized: {"class": "Vector3D", "args": [10, -7, 3]}
```

```
After:       Vector3D(10, -7, 3)
```

```
class Field(object):
    def __init__(self, name):
        self.name = name
        self.internal_name = '_' + self.name

    def __get__(self, instance, instance_type):
        if instance is None: return self
        return getattr(instance, self.internal_name, '')

    def __set__(self, instance, value):
        setattr(instance, self.internal_name, value)
```

```
class Customer(object):  
    # Class attributes  
    first_name = Field('first_name')  
    last_name = Field('last_name')  
    prefix = Field('prefix')  
    suffix = Field('suffix')
```

```
foo = Customer()
print('Before:', repr(foo.first_name), foo.__dict__)
foo.first_name = 'Euclid'
print('After: ', repr(foo.first_name), foo.__dict__)

>>>
Before: '' {}
After: 'Euclid' {'_first_name': 'Euclid'}
```

```
class Meta(type):
    def __new__(meta, name, bases, class_dict):
        for key, value in class_dict.items():
            if isinstance(value, Field):
                value.name = key
                value.internal_name = '_' + key
        cls = type.__new__(meta, name, bases, class_dict)
        return cls
```

```
class DatabaseRow(object, metaclass=Meta):  
    pass
```

```
class Field(object):  
    def __init__(self):  
        # These will be assigned by the metaclass.  
        self.name = None  
        self.internal_name = None  
    # ...
```

```
class BetterCustomer(DatabaseRow):  
    first_name = Field()  
    last_name = Field()  
    prefix = Field()  
    suffix = Field()
```



```
foo = BetterCustomer()
print('Before:', repr(foo.first_name), foo.__dict__)
foo.first_name = 'Euler'
print('After: ', repr(foo.first_name), foo.__dict__)

>>>
Before: '' {}
After: 'Euler' {'_first_name': 'Euler'}
```

```
proc = subprocess.Popen(  
    ['echo', 'Hello from the child!'],  
    stdout=subprocess.PIPE)  
out, err = proc.communicate()  
print(out.decode('utf-8'))
```

```
>>>
```

```
Hello from the child!
```

```
proc = subprocess.Popen(['sleep', '0.3'])
while proc.poll() is None:
    print('Working...')
    # Some time-consuming work here
    # ...

print('Exit status', proc.poll())

>>>
Working...
Working...
Exit status 0
```

```
def run_sleep(period):  
    proc = subprocess.Popen(['sleep', str(period)])  
    return proc  
  
start = time()  
procs = []  
for _ in range(10):  
    proc = run_sleep(0.1)  
    procs.append(proc)
```

```
for proc in procs:
    proc.communicate()
end = time()
print('Finished in %.3f seconds' % (end - start))

>>>
Finished in 0.117 seconds
```

```
def run_openssl(data):  
    env = os.environ.copy()  
    env['password'] = b'\xe24U\n\xd0Q13S\x11'  
    proc = subprocess.Popen(  
        ['openssl', 'enc', '-des3', '-pass', 'env:password'],  
        env=env,  
        stdin=subprocess.PIPE,  
        stdout=subprocess.PIPE)  
    proc.stdin.write(data)  
    proc.stdin.flush() # Ensure the child gets input  
    return proc
```

```
for proc in procs:
    out, err = proc.communicate()
    print(out[-10:])

>>>
b'o4,G\x91\x95\xfe\xa0\xaa\xb7'
b'\x0b\x01\\\xb1\xb7\xfb\xb2C\xe1b'
b'ds\xc5\xf4;j\x1f\xd0c-'
```

```
def run_md5(input_stdin):  
    proc = subprocess.Popen(  
        ['md5'],  
        stdin=input_stdin,  
        stdout=subprocess.PIPE)  
    return proc
```



```
input_procs = []  
hash_procs = []  
for _ in range(3):  
    data = os.urandom(10)  
    proc = run_openssl(data)  
    input_procs.append(proc)  
    hash_proc = run_md5(proc.stdout)  
    hash_procs.append(hash_proc)
```

```
for proc in input_procs:
    proc.communicate()
for proc in hash_procs:
    out, err = proc.communicate()
    print(out.strip())
```

```
>>>
```

```
b'7a1822875dcf9650a5a71e5e41e77bf3'
b'd41d8cd98f00b204e9800998ecf8427e'
b'1720f581cfdc448b6273048d42621100'
```

```
proc = run_sleep(10)
try:
    proc.communicate(timeout=0.1)
except subprocess.TimeoutExpired:
    proc.terminate()
    proc.wait()

print('Exit status', proc.poll())

>>>
Exit status -15
```

```
def factorize(number):  
    for i in range(1, number + 1):  
        if number % i == 0:  
            yield i
```

```
numbers = [2139079, 1214759, 1516637, 1852285]
start = time()
for number in numbers:
    list(factorize(number))
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 1.040 seconds
```

```
from threading import Thread
```

```
class FactorizeThread(Thread):
```

```
    def __init__(self, number):
```

```
        super().__init__()
```

```
        self.number = number
```

```
    def run(self):
```

```
        self.factors = list(factorize(self.number))
```

```
start = time()
threads = []
for number in numbers:
    thread = FactorizeThread(number)
    thread.start()
    threads.append(thread)
```

```
for thread in threads:
    thread.join()
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 1.061 seconds
```



```
import select
```

```
def slow_systemcall():  
    select.select([], [], [], 0.1)
```

```
start = time()
for _ in range(5):
    slow_systemcall()
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 0.503 seconds
```

```
start = time()
threads = []
for _ in range(5):
    thread = Thread(target=slow_systemcall)
    thread.start()
    threads.append(thread)
```

```
def compute_helicopter_location(index):  
    # ...  
  
for i in range(5):  
    compute_helicopter_location(i)  
for thread in threads:  
    thread.join()  
end = time()  
print('Took %.3f seconds' % (end - start))  
  
>>>  
Took 0.102 seconds
```

```
class Counter(object):  
    def __init__(self):  
        self.count = 0  
  
    def increment(self, offset):  
        self.count += offset
```

```
def worker(sensor_index, how_many, counter):  
    for _ in range(how_many):  
        # Read from the sensor  
        # ...  
        counter.increment(1)
```

```
def run_threads(func, how_many, counter):  
    threads = []  
    for i in range(5):  
        args = (i, how_many, counter)  
        thread = Thread(target=func, args=args)  
        threads.append(thread)  
        thread.start()  
    for thread in threads:  
        thread.join()
```

```
how_many = 10**5  
counter = Counter()  
run_threads(worker, how_many, counter)  
print('Counter should be %d, found %d' %  
      (5 * how_many, counter.count))
```

```
>>>
```

```
Counter should be 500000, found 278328
```



```
value = getattr(counter, 'count')  
result = value + offset  
setattr(counter, 'count', result)
```

```
# Running in Thread A
value_a = getattr(counter, 'count')
# Context switch to Thread B
value_b = getattr(counter, 'count')
result_b = value_b + 1
setattr(counter, 'count', result_b)
# Context switch back to Thread A
result_a = value_a + 1
setattr(counter, 'count', result_a)
```

```
class LockingCounter(object):  
    def __init__(self):  
        self.lock = Lock()  
        self.count = 0  
  
    def increment(self, offset):  
        with self.lock:  
            self.count += offset
```

```
counter = LockingCounter()
run_threads(worker, how_many, counter)
print('Counter should be %d, found %d' %
      (5 * how_many, counter.count))
>>>
Counter should be 500000, found 500000
```

```
def put(self, item):  
    with self.lock:  
        self.items.append(item)
```

```
def get(self):  
    with self.lock:  
        return self.items.popleft()
```

```
class Worker(Thread):  
    def __init__(self, func, in_queue, out_queue):  
        super().__init__()  
        self.func = func  
        self.in_queue = in_queue  
        self.out_queue = out_queue  
        self.polled_count = 0  
        self.work_done = 0
```

```
def run(self):  
    while True:  
        self.polled_count += 1  
        try:  
            item = self.in_queue.get()  
        except IndexError:  
            sleep(0.01) # No work to do  
        else:  
            result = self.func(item)  
            self.out_queue.put(result)  
            self.work_done += 1
```



```
download_queue = MyQueue()
resize_queue = MyQueue()
upload_queue = MyQueue()
done_queue = MyQueue()
threads = [
    Worker(download, download_queue, resize_queue),
    Worker(resize, resize_queue, upload_queue),
    Worker(upload, upload_queue, done_queue),
]
```

```
for thread in threads:  
    thread.start()  
for _ in range(1000):  
    download_queue.put(object())
```

```
while len(done_queue.items) < 1000:  
    # Do something useful while waiting  
    # ...
```

```
processed = len(done_queue.items)
polled = sum(t.polled_count for t in threads)
print('Processed', processed, 'items after polling',
      polled, 'times')
```

```
>>>
```

```
Processed 1000 items after polling 3030 times
```

```
from queue import Queue
queue = Queue()

def consumer():
    print('Consumer waiting')
    queue.get() # Runs after put() below
    print('Consumer done')

thread = Thread(target=consumer)
thread.start()
```

```
print('Producer putting')
queue.put(object())
thread.join()
print('Producer done')
```

Runs before get() above

```
>>>
```

```
Consumer waiting
Producer putting
Consumer done
Producer done
```

```
queue = Queue(1)                                # Buffer size of 1

def consumer():
    time.sleep(0.1)                               # Wait
    queue.get()                                   # Runs second
    print('Consumer got 1')
    queue.get()                                   # Runs fourth
    print('Consumer got 2')

thread = Thread(target=consumer)
thread.start()
```

```
queue.put(object())           # Runs first
print('Producer put 1')
queue.put(object())           # Runs third
print('Producer put 2')
thread.join()
print('Producer done')
```

```
>>>
```

```
Producer put 1
Consumer got 1
Producer put 2
Consumer got 2
Producer done
```



```
in_queue = Queue()
```

```
def consumer():  
    print('Consumer waiting')  
    work = in_queue.get()          # Done second  
    print('Consumer working')  
    # Doing work  
    # ...  
    print('Consumer done')  
    in_queue.task_done()          # Done third
```

```
Thread(target=consumer).start()
```

```
in_queue.put(object())  
print('Producer waiting')  
in_queue.join()  
print('Producer done')
```

Done first

Done fourth

```
>>>
```

```
Consumer waiting
```

```
Producer waiting
```

```
Consumer working
```

```
Consumer done
```

```
Producer done
```

```
class ClosableQueue(Queue):  
    SENTINEL = object()  
  
    def close(self):  
        self.put(self.SENTINEL)
```

```
def __iter__(self):  
    while True:  
        item = self.get()  
        try:  
            if item is self.SENTINEL:  
                return # Cause the thread to exit  
            yield item  
        finally:  
            self.task_done()
```

```
class StoppableWorker(Thread):
    def __init__(self, func, in_queue, out_queue):
        # ...

    def run(self):
        for item in self.in_queue:
            result = self.func(item)
            self.out_queue.put(result)
```

```
download_queue = ClosableQueue()
# ...
threads = [
    StoppableWorker(download, download_queue, resize_queue),
    # ...
]
```

```
for thread in threads:  
    thread.start()  
for _ in range(1000):  
    download_queue.put(object())  
download_queue.close()
```

```
download_queue.join()
resize_queue.close()
resize_queue.join()
upload_queue.close()
upload_queue.join()
print(done_queue.qsize(), 'items finished')

>>>
1000 items finished
```



```
def my_coroutine():
    while True:
        received = yield
        print('Received:', received)

it = my_coroutine()
next(it)                # Prime the coroutine
it.send('First')
it.send('Second')

>>>
Received: First
Received: Second
```

```
def minimize():  
    current = yield  
    while True:  
        value = yield current  
        current = min(value, current)
```

```
it = minimize()  
next(it)                                # Prime the generator  
print(it.send(10))  
print(it.send(4))  
print(it.send(22))  
print(it.send(-1))  
  
>>>  
10  
4  
4  
-1
```

0	1	2	3	4
-----	-----	-----	-----	-----
-*- - -	--* - -	--** -	--* - -	- - - - -
- - ** -	- - ** -	- * - - -	- * - - -	- ** - -
- - - * -	- - ** -	- - ** -	- - * - -	- - - - -
-----	-----	-----	-----	-----

```
Query = namedtuple('Query', ('y', 'x'))
```

```
def count_neighbors(y, x):
    n_ = yield Query(y + 1, x + 0) # North
    ne = yield Query(y + 1, x + 1) # Northeast
    # Define e_, se, s_, sw, w_, nw ...
    # ...
    neighbor_states = [n_, ne, e_, se, s_, sw, w_, nw]
    count = 0
    for state in neighbor_states:
        if state == ALIVE:
            count += 1
    return count
```

```

it = count_neighbors(10, 5)
q1 = next(it)           # Get the first query
print('First yield: ', q1)
q2 = it.send(ALIVE)      # Send q1 state, get q2
print('Second yield:', q2)
q3 = it.send(ALIVE)      # Send q2 state, get q3
# ...
try:
    count = it.send(EMPTY) # Send q8 state, retrieve count
except StopIteration as e:
    print('Count: ', e.value) # Value from return statement
>>>
First yield:  Query(y=11, x=5)
Second yield: Query(y=11, x=6)
...
Count:  2

```

```
Transition = namedtuple('Transition', ('y', 'x', 'state'))
```



```
def game_logic(state, neighbors):  
    # ...  
  
def step_cell(y, x):  
    state = yield Query(y, x)  
    neighbors = yield from count_neighbors(y, x)  
    next_state = game_logic(state, neighbors)  
    yield Transition(y, x, next_state)
```

```
def game_logic(state, neighbors):  
    if state == ALIVE:  
        if neighbors < 2:  
            return EMPTY        # Die: Too few  
        elif neighbors > 3:  
            return EMPTY        # Die: Too many  
    else:  
        if neighbors == 3:  
            return ALIVE        # Regenerate  
    return state
```

```

it = step_cell(10, 5)
q0 = next(it)           # Initial location query
print('Me:      ', q0)
q1 = it.send(ALIVE)      # Send my status, get neighbor query
print('Q1:      ', q1)
# ...
t1 = it.send(EMPTY)      # Send for q8, get game decision
print('Outcome: ', t1)

>>>
Me:      Query(y=10, x=5)
Q1:      Query(y=11, x=5)
...
Outcome: Transition(y=10, x=5, state='-')

```

```
TICK = object()
```

```
def simulate(height, width):  
    while True:  
        for y in range(height):  
            for x in range(width):  
                yield from step_cell(y, x)  
            yield TICK
```

```
class Grid(object):
    def __init__(self, height, width):
        self.height = height
        self.width = width
        self.rows = []
        for _ in range(self.height):
            self.rows.append([EMPTY] * self.width)

    def __str__(self):
        # ...
```

```
def query(self, y, x):  
    return self.rows[y % self.height][x % self.width]  
  
def assign(self, y, x, state):  
    self.rows[y % self.height][x % self.width] = state
```

```
def live_a_generation(grid, sim):  
    progeny = Grid(grid.height, grid.width)  
    item = next(sim)  
    while item is not TICK:  
        if isinstance(item, Query):  
            state = grid.query(item.y, item.x)  
            item = sim.send(state)  
        else: # Must be a Transition  
            progeny.assign(item.y, item.x, item.state)  
            item = next(sim)  
    return progeny
```

```

class ColumnPrinter(object):
    # ...

columns = ColumnPrinter()
sim = simulate(grid.height, grid.width)
for i in range(5):
    columns.append(str(grid))
    grid = live_a_generation(grid, sim)

print(columns)

```

```
>>>
```

0	1	2	3	4
---*-----	-----	-----	-----	-----
---*-----	--*_*-----	---*-----	---*-----	---*-----
--***-----	---**-----	--*_*-----	---**-----	---*-----
-----	---*-----	---**-----	---**-----	---***-----
-----	-----	-----	-----	-----


```
# Python 2
def delegated():
    yield 1
    yield 2

def composed():
    yield 'A'
    for value in delegated(): # yield from in Python 3
        yield value
    yield 'B'

print list(composed())

>>>
['A', 1, 2, 'B']
```

```
# Python 2
```

```
class MyReturn(Exception):  
    def __init__(self, value):  
        self.value = value  
  
def delegated():  
    yield 1  
    raise MyReturn(2) # return 2 in Python 3  
    yield 'Not reached'  
  
def composed():  
    try:  
        for value in delegated():  
            yield value  
    except MyReturn as e:  
        output = e.value  
        yield output * 4  
  
print list(composed())  
  
>>>  
[1, 8]
```

```
def gcd(pair):  
    a, b = pair  
    low = min(a, b)  
    for i in range(low, 0, -1):  
        if a % i == 0 and b % i == 0:  
            return i
```

```
numbers = [(1963309, 2265973), (2030677, 3814172),  
           (1551645, 2229620), (2039045, 2020802)]  
start = time()  
results = list(map(gcd, numbers))  
end = time()  
print('Took %.3f seconds' % (end - start))  
  
>>>  
Took 1.170 seconds
```

```
start = time()
pool = ThreadPoolExecutor(max_workers=2)
results = list(pool.map(gcd, numbers))
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 1.199 seconds
```

```
start = time()
pool = ProcessPoolExecutor(max_workers=2) # The one change
results = list(pool.map(gcd, numbers))
end = time()
print('Took %.3f seconds' % (end - start))

>>>
Took 0.663 seconds
```

```
def trace(func):  
    def wrapper(*args, **kwargs):  
        result = func(*args, **kwargs)  
        print('%s(%r, %r) -> %r' %  
              (func.__name__, args, kwargs, result))  
        return result  
    return wrapper
```

@trace

def fibonacci(n):

"""Return the n-th Fibonacci number"""

if n in (0, 1):

return n

return (fibonacci(n - 2) + fibonacci(n - 1))


```
print(fibonacci)
```

```
>>>
```

```
<function trace.<locals>.wrapper at 0x107f7ed08>
```

```
help(fibonacci)
```

```
>>>
```

```
Help on function wrapper in module __main__:
```

```
wrapper(*args, **kwargs)
```

```
def trace(func):  
    @wraps(func)  
    def wrapper(*args, **kwargs):  
        # ...  
    return wrapper
```

```
@trace  
def fibonacci(n):  
    # ...
```

```
help(fibonacci)
```

```
>>>
```

```
Help on function fibonacci in module __main__:
```

```
fibonacci(n)
```

```
    Return the n-th Fibonacci number
```

```
def my_function():  
    logging.debug('Some debug data')  
    logging.error('Error log here')  
    logging.debug('More debug data')
```

```
@contextmanager
def debug_logging(level):
    logger = logging.getLogger()
    old_level = logger.getEffectiveLevel()
    logger.setLevel(level)
    try:
        yield
    finally:
        logger.setLevel(old_level)
```

```
with debug_logging(logging.DEBUG):  
    print('Inside:')  
    my_function()  
print('After:')  
my_function()
```

```
>>>
```

```
Inside:
```

```
Some debug data
```

```
Error log here
```

```
More debug data
```

```
After:
```

```
Error log here
```

```
with open('/tmp/my_output.txt', 'w') as handle:  
    handle.write('This is some data!')
```



```
@contextmanager
def log_level(level, name):
    logger = logging.getLogger(name)
    old_level = logger.getEffectiveLevel()
    logger.setLevel(level)
    try:
        yield logger
    finally:
        logger.setLevel(old_level)
```

```
with log_level(logging.DEBUG, 'my-log') as logger:  
    logger.debug('This is my message!')  
    logging.debug('This will not print')
```

```
>>>
```

```
This is my message!
```

```
logger = logging.getLogger('my-log')  
logger.debug('Debug will not print')  
logger.error('Error will print')
```

```
>>>
```

```
Error will print
```

```
state = GameState()  
state.level += 1 # Player beat a level  
state.lives -= 1 # Player had to try again
```

```
state_path = '/tmp/game_state.bin'  
with open(state_path, 'wb') as f:  
    pickle.dump(state, f)
```

```
with open(state_path, 'rb') as f:  
    state_after = pickle.load(f)  
print(state_after.__dict__)
```

```
>>>
```

```
{'lives': 3, 'level': 1}
```

```
state = GameState()
serialized = pickle.dumps(state)
state_after = pickle.loads(serialized)
print(state_after.__dict__)
```

```
>>>
```

```
{'lives': 4, 'level': 0, 'points': 0}
```

```
with open(state_path, 'rb') as f:  
    state_after = pickle.load(f)  
print(state_after.__dict__)
```

```
>>>
```

```
{'lives': 3, 'level': 1}
```



```
assert isinstance(state_after, GameState)
```

```
class GameState(object):  
    def __init__(self, level=0, lives=4, points=0):  
        self.level = level  
        self.lives = lives  
        self.points = points
```

```
def pickle_game_state(game_state):  
    kwargs = game_state.__dict__  
    return unpickle_game_state, (kwargs,)
```

```
def unpickle_game_state(kwargs):  
    return GameState(**kwargs)
```

```
copyreg.pickle(GameState, pickle_game_state)
```

```
state = GameState()
state.points += 1000
serialized = pickle.dumps(state)
state_after = pickle.loads(serialized)
print(state_after.__dict__)

>>>
{'lives': 4, 'level': 0, 'points': 1000}
```

```
class GameState(object):  
    def __init__(self, level=0, lives=4, points=0, magic=5):  
        # ...
```

```
state_after = pickle.loads(serialized)
print(state_after.__dict__)

>>>
{'level': 0, 'points': 1000, 'magic': 5, 'lives': 4}
```



```
class GameState(object):  
    def __init__(self, level=0, points=0, magic=5):  
        # ...
```

```
pickle.loads(serialized)
```

```
>>>
```

```
TypeError: __init__() got an unexpected keyword argument  
↳ 'lives'
```

```
def pickle_game_state(game_state):  
    kwargs = game_state.__dict__  
    kwargs['version'] = 2  
    return unpickle_game_state, (kwargs,)
```

```
def unpickle_game_state(kwargs):  
    version = kwargs.pop('version', 1)  
    if version == 1:  
        kwargs.pop('lives')  
    return GameState(**kwargs)
```

```
copyreg.pickle(GameState, pickle_game_state)
state_after = pickle.loads(serialized)
print(state_after.__dict__)
```

```
>>>
```

```
{'magic': 5, 'level': 0, 'points': 1000}
```

```
class BetterGameState(object):  
    def __init__(self, level=0, points=0, magic=5):  
        # ...
```

```
pickle.loads(serialized)
```

```
>>>
```

```
AttributeError: Can't get attribute 'GameState' on <module  
↳ '__main__' from 'my_code.py'>
```

```
print(serialized[:25])
```

```
>>>
```

```
b'\x80\x03c__main__\nGameState\nq\x00) '
```



```
copyreg.pickle(BetterGameState, pickle_game_state)
```

```
state = BetterGameState()
serialized = pickle.dumps(state)
print(serialized[:35])

>>>
b'\x80\x03c__main__\nunpickle_game_state\nq\x00{'
```

```
from time import localtime, strftime  
  
now = 1407694710  
local_tuple = localtime(now)  
time_format = '%Y-%m-%d %H:%M:%S'  
time_str = strftime(time_format, local_tuple)  
print(time_str)  
  
>>>  
2014-08-10 11:18:30
```

```
from time import mktime, strptime

time_tuple = strptime(time_str, time_format)
utc_now = mktime(time_tuple)
print(utc_now)

>>>
1407694710.0
```

```
parse_format = '%Y-%m-%d %H:%M:%S %Z'
depart_sfo = '2014-05-01 15:45:16 PDT'
time_tuple =.strptime(depart_sfo, parse_format)
time_str = strftime(time_format, time_tuple)
print(time_str)
```

```
>>>
```

```
2014-05-01 15:45:16
```

```
arrival_nyc = '2014-05-01 23:33:24 EDT'  
time_tuple =.strptime(arrival_nyc, time_format)  
  
>>>  
ValueError: unconverted data remains:  EDT
```

```
from datetime import datetime, timezone  
  
now = datetime(2014, 8, 10, 18, 18, 30)  
now_utc = now.replace(tzinfo=timezone.utc)  
now_local = now_utc.astimezone()  
print(now_local)
```

```
>>>
```

```
2014-08-10 11:18:30-07:00
```

```
time_str = '2014-08-10 11:18:30'  
now = datetime.strptime(time_str, time_format)  
time_tuple = now.timetuple()  
utc_now = mktime(time_tuple)  
print(utc_now)
```

```
>>>
```

```
1407694710.0
```



```
arrival_nyc = '2014-05-01 23:33:24'
nyc_dt_naive = datetime.strptime(arrival_nyc, time_format)
eastern = pytz.timezone('US/Eastern')
nyc_dt = eastern.localize(nyc_dt_naive)
utc_dt = pytz.utc.normalize(nyc_dt.astimezone(pytz.utc))
print(utc_dt)
```

```
>>>
```

```
2014-05-02 03:33:24+00:00
```

```
pacific = pytz.timezone('US/Pacific')  
sf_dt = pacific.normalize(utc_dt.astimezone(pacific))  
print(sf_dt)
```

```
>>>
```

```
2014-05-01 20:33:24-07:00
```

```
nepal = pytz.timezone('Asia/Katmandu')  
nepal_dt = nepal.normalize(utc_dt.astimezone(nepal))  
print(nepal_dt)
```

```
>>>
```

```
2014-05-02 09:18:24+05:45
```

```
fifo = deque()  
fifo.append(1)           # Producer  
x = fifo.popleft()       # Consumer
```

```
a = {}  
a['foo'] = 1  
a['bar'] = 2
```

Randomly populate 'b' to cause hash conflicts

```
while True:  
    z = randint(99, 1013)  
    b = {}  
    for i in range(z):  
        b[i] = i  
    b['foo'] = 1  
    b['bar'] = 2  
    for i in range(z):  
        del b[i]  
    if str(b) != str(a):  
        break
```

```
print(a)  
print(b)  
print('Equal?', a == b)
```

```
>>>  
{'foo': 1, 'bar': 2}  
{'bar': 2, 'foo': 1}  
Equal? True
```

```
a = OrderedDict()
a['foo'] = 1
a['bar'] = 2
b = OrderedDict()
b['foo'] = 'red'
b['bar'] = 'blue'

for value1, value2 in zip(a.values(), b.values()):
    print(value1, value2)

>>>
1 red
2 blue
```

```
print(heapop(a), heapop(a), heapop(a), heapop(a))  
>>>  
3 4 5 7
```

```
a = []  
heappush(a, 5)  
heappush(a, 3)  
heappush(a, 7)  
heappush(a, 4)  
assert a[0] == nsmaallest(1, a)[0] == 3
```



```
rate = Decimal('1.45')
seconds = Decimal('222') # 3*60 + 42
cost = rate * seconds / Decimal('60')
print(cost)
```

```
>>>
```

```
5.365
```

```
rounded = cost.quantize(Decimal('0.01'), rounding=ROUND_UP)
print(rounded)
```

```
>>>
```

```
5.37
```

```
rate = Decimal('0.05')
seconds = Decimal('5')
cost = rate * seconds / Decimal('60')
print(cost)
```

>>>

0.00416666666666666666666666666667

```
rounded = cost.quantize(Decimal('0.01'), rounding=ROUND_UP)
print(rounded)
```

```
>>>
```

```
0.01
```

```
$ pip3 install pytz
```

```
Downloading/unpacking pytz
```

```
  Downloading pytz-2014.4.tar.bz2 (159kB): 159kB downloaded
```

```
  Running setup.py (...) egg_info for package pytz
```

```
Installing collected packages: pytz
```

```
  Running setup.py install for pytz
```

```
Successfully installed pytz
```

```
Cleaning up...
```

```
def palindrome(word):  
    """Return True if the given word is a palindrome."""  
    return word == word[::-1]
```

```
print(repr(palindrome.__doc__))
```

```
>>>
```

```
'Return True if the given word is a palindrome.'
```

```
# words.py
#!/usr/bin/env python3
"""Library for testing words for various linguistic patterns.

Testing how words relate to each other can be tricky sometimes!
This module provides easy ways to determine when words you've
found have special properties.

Available functions:
- palindrome: Determine if a word is a palindrome.
- check_anagram: Determine if two words are anagrams.
...
"""

# ...
```



```
class Player(object):
    """Represents a player of the game.

    Subclasses may override the 'tick' method to provide
    custom animations for the player's movement depending
    on their power level, etc.

    Public attributes:
    - power: Unused power-ups (float between 0 and 1).
    - coins: Coins found during the level (integer).
    """

    # ...
```

```
def find_anagrams(word, dictionary):  
    """Find all anagrams for a word.
```

This function only runs as fast as the test for membership in the 'dictionary' container. It will be slow if the dictionary is a list and fast if it's a set.

Args:

word: String of the target word.
dictionary: Container with all strings that are known to be actual words.

Returns:

List of anagrams that were found. Empty if none were found.

```
    """
```

```
# ...
```

```
# main.py
```

```
from analysis.utils import log_base2_bucket  
from frontend.utils import stringify
```

```
bucket = stringify(log_base2_bucket(33))
```

```
# main2.py
from analysis.utils import inspect
from frontend.utils import inspect # Overwrites!
```

```
# main3.py
from analysis.utils import inspect as analysis_inspect
from frontend.utils import inspect as frontend_inspect

value = 33
if analysis_inspect(value) == frontend_inspect(value):
    print('Inspection equal!')
```

```
# models.py
```

```
__all__ = ['Projectile']
```

```
class Projectile(object):
```

```
    def __init__(self, mass, velocity):
```

```
        self.mass = mass
```

```
        self.velocity = velocity
```

```
# utils.py
from . models import Projectile

__all__ = ['simulate_collision']

def _dot_product(a, b):
    # ...

def simulate_collision(a, b):
    # ...
```

```
# api_consumer.py
from mypackage import *

a = Projectile(1.5, 3)
b = Projectile(4, 1.7)
after_a, after_b = simulate_collision(a, b)
```



```
def determine_weight(volume, density):  
    if density <= 0:  
        raise ValueError('Density must be positive')  
    # ...
```

```
# my_module.py
class Error(Exception):
    """Base-class for all exceptions raised by this module."""

class InvalidDensityError(Error):
    """There was a problem with a provided density value."""
```

```
try:
    weight = my_module.determine_weight(1, -1)
except my_module.Error as e:
    logging.error('Unexpected error: %s', e)
```

```
try:
    weight = my_module.determine_weight(1, -1)
except my_module.InvalidDensityError:
    weight = 0
except my_module.Error as e:
    logging.error('Bug in the calling code: %s', e)
```

```
try:
    weight = my_module.determine_weight(1, -1)
except my_module.InvalidDensityError:
    weight = 0
except my_module.Error as e:
    logging.error('Bug in the calling code: %s', e)
except Exception as e:
    logging.error('Bug in the API code: %s', e)
    raise
```

```
# my_module.py
class NegativeDensityError(InvalidDensityError):
    """A provided density value was negative."""

def determine_weight(volume, density):
    if density < 0:
        raise NegativeDensityError
```

```
try:
    weight = my_module.determine_weight(1, -1)
except my_module.NegativeDensityError as e:
    raise ValueError('Must supply non-negative density') from e
except my_module.InvalidDensityError:
    weight = 0
except my_module.Error as e:
    logging.error('Bug in the calling code: %s', e)
except Exception as e:
    logging.error('Bug in the API code: %s', e)
    raise
```

```
# my_module.py
class WeightError(Error):
    """Base-class for weight calculation errors."""

class VolumeError(Error):
    """Base-class for volume calculation errors."""

class DensityError(Error):
    """Base-class for density calculation errors."""
```



```
# dialog.py
```

```
import app
```

```
class Dialog(object):
```

```
    def __init__(self, save_dir):
```

```
        self.save_dir = save_dir
```

```
    # ...
```

```
save_dialog = Dialog(app.prefs.get('save_dir'))
```

```
def show():
```

```
    # ...
```

```
Traceback (most recent call last):
  File "main.py", line 4, in <module>
    import app
  File "app.py", line 4, in <module>
    import dialog
  File "dialog.py", line 16, in <module>
    save_dialog = Dialog(app.prefs.get('save_dir'))
AttributeError: 'module' object has no attribute 'prefs'
```

```
# dialog.py
import app

class Dialog(object):
    # ...

save_dialog = Dialog()

def show():
    # ...

def configure():
    save_dialog.save_dir = app.prefs.get('save_dir')
```

```
# dialog.py
class Dialog(object):
    # ...

save_dialog = Dialog()

def show():
    import app # Dynamic import
    save_dialog.save_dir = app.prefs.get('save_dir')
    # ...
```

```
$ pip3 show Sphinx
```

```
---
```

```
Name: Sphinx
```

```
Version: 1.2.2
```

```
Location: /usr/local/lib/python3.4/site-packages
```

```
Requires: docutils, Jinja2, Pygments
```

```
$ pip3 show flask
```

```
---
```

```
Name: Flask
```

```
Version: 0.10.1
```

```
Location: /usr/local/lib/python3.4/site-packages
```

```
Requires: Werkzeug, Jinja2, itsdangerous
```

```
$ pyvenv /tmp/myproject
```

```
$ cd /tmp/myproject
```

```
$ ls
```

```
bin      include  lib      pyvenv.cfg
```

```
(myproject)$ which python3
/tmp/myproject/bin/python3
(myproject)$ ls -l /tmp/myproject/bin/python3
... -> /tmp/myproject/bin/python3.4
(myproject)$ ls -l /tmp/myproject/bin/python3.4
... -> /usr/local/bin/python3.4
```



```
(myproject)$ python3 -c 'import pytz'
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ImportError: No module named 'pytz'
```

```
(myproject)$ pip3 install pytz
```

```
(myproject)$ python3 -c 'import pytz'  
(myproject)$
```

```
(myproject)$ pip3 freeze > requirements.txt  
(myproject)$ cat requirements.txt  
numpy==1.8.2  
pytz==2014.4  
requests==2.3.0
```

```
(otherproject)$ pip3 install -r /tmp/myproject/requirements.txt
```

```
# dev_main.py
```

```
TESTING = True
```

```
import db_connection
```

```
db = db_connection.Database()
```

```
# prod_main.py
```

```
TESTING = False
```

```
import db_connection
```

```
db = db_connection.Database()
```

```
# db_connection.py
import __main__

class TestingDatabase(object):
    # ...

class RealDatabase(object):
    # ...

if __main__.TESTING:
    Database = TestingDatabase
else:
    Database = RealDatabase
```

```
# db_connection.py
import sys

class Win32Database(object):
    # ...

class PosixDatabase(object):
    # ...

if sys.platform.startswith('win32'):
    Database = Win32Database
else:
    Database = PosixDatabase
```



```
class OpaqueClass(object):  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y
```

```
obj = OpaqueClass(1, 2)  
print(obj)
```

```
>>>
```

```
<__main__.OpaqueClass object at 0x107880ba8>
```

```
class BetterClass(object):  
    def __init__(self, x, y):  
        # ...  
  
    def __repr__(self):  
        return 'BetterClass(%d, %d)' % (self.x, self.y)
```

```
# utils.py
def to_str(data):
    if isinstance(data, str):
        return data
    elif isinstance(data, bytes):
        return data.decode('utf-8')
    else:
        raise TypeError('Must supply str or bytes, '
                        'found: %r' % data)
```

```
# utils_test.py
from unittest import TestCase, main
from utils import to_str

class UtilsTestCase(TestCase):
    def test_to_str_bytes(self):
        self.assertEqual('hello', to_str(b'hello'))

    def test_to_str_str(self):
        self.assertEqual('hello', to_str('hello'))

    def test_to_str_bad(self):
        self.assertRaises(TypeError, to_str, object())

if __name__ == '__main__':
    main()
```

```
class MyTest(TestCase):
    def setUp(self):
        self.test_dir = TemporaryDirectory()
    def tearDown(self):
        self.test_dir.cleanup()
    # Test methods follow
    # ...
```

```
def complex_func(a, b, c):  
    # ...  
    import pdb; pdb.set_trace()
```

```
-> import pdb; pdb.set_trace()  
(Pdb)
```

```
def insertion_sort(data):  
    result = []  
    for value in data:  
        insert_value(result, value)  
    return result
```



```
def insert_value(array, value):  
    for i, existing in enumerate(array):  
        if existing > value:  
            array.insert(i, value)  
            return  
    array.append(value)
```

```
from random import randint
```

```
max_size = 10**4
```

```
data = [randint(0, max_size) for _ in range(max_size)]
```

```
test = lambda: insertion_sort(data)
```

```
>>>
```

```
20003 function calls in 1.812 seconds
```

```
Ordered by: cumulative time
```

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	1.812	1.812	main.py:34(<lambda>)
1	0.003	0.003	1.812	1.812	main.py:10(insertion_sort)
10000	1.797	0.000	1.810	0.000	main.py:20(insert_value)
9992	0.013	0.000	0.013	0.000	{method 'insert' of 'list' objects}
8	0.000	0.000	0.000	0.000	{method 'append' of 'list' objects}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

```
from bisect import bisect_left

def insert_value(array, value):
    i = bisect_left(array, value)
    array.insert(i, value)
```

```
>>>
```

```
30003 function calls in 0.028 seconds
```

```
Ordered by: cumulative time
```

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.028	0.028	main.py:34(<lambda>)
1	0.002	0.002	0.028	0.028	main.py:10(insertion_sort)
10000	0.005	0.000	0.026	0.000	main.py:112(insert_value)
10000	0.014	0.000	0.014	0.000	{method 'insert' of 'list' objects}
10000	0.007	0.000	0.007	0.000	{built-in method bisect_left}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

```
>>>
```

```
20242 function calls in 0.208 seconds
```

```
Ordered by: cumulative time
```

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	0.208	0.208	main.py:176(my_program)
20	0.005	0.000	0.206	0.010	main.py:168(first_func)
20200	0.203	0.000	0.203	0.000	main.py:161(my_utility)
20	0.000	0.000	0.002	0.000	main.py:172(second_func)
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

```
>>>
```

```
Ordered by: cumulative time
```

Function	was called by...	ncalls	tottime	cumtime
main.py:176(my_program)	<-			
main.py:168(first_func)	<-	20	0.005	0.206
main.py:161(my_utility)	<-	20000	0.202	0.202
		200	0.002	0.002
main.py:172(second_func)	<-	20	0.000	0.002

```
# using_gc.py
import gc
found_objects = gc.get_objects()
print('%d objects before' % len(found_objects))

import waste_memory
x = waste_memory.run()
found_objects = gc.get_objects()
print('%d objects after' % len(found_objects))
for obj in found_objects[:3]:
    print(repr(obj)[:100])

>>>
4756 objects before
14873 objects after
<waste_memory.MyObject object at 0x1063f6940>
<waste_memory.MyObject object at 0x1063f6978>
<waste_memory.MyObject object at 0x1063f69b0>
```



```
# top_n.py
import tracemalloc
tracemalloc.start(10) # Save up to 10 stack frames

time1 = tracemalloc.take_snapshot()
import waste_memory
x = waste_memory.run()
time2 = tracemalloc.take_snapshot()

stats = time2.compare_to(time1, 'lineno')
for stat in stats[:3]:
    print(stat)

>>>
waste_memory.py:6: size=2235 KiB (+2235 KiB), count=29981 (+29981), average=76 B
waste_memory.py:7: size=869 KiB (+869 KiB), count=10000 (+10000), average=89 B
waste_memory.py:12: size=547 KiB (+547 KiB), count=10000 (+10000), average=56 B
```

```
# with_trace.py
# ...
stats = time2.compare_to(time1, 'traceback')
top = stats[0]
print('\n'.join(top.traceback.format()))

>>>
File "waste_memory.py", line 6
    self.x = os.urandom(100)
File "waste_memory.py", line 12
    obj = MyObject()
File "waste_memory.py", line 19
    deep_values.append(get_data())
File "with_trace.py", line 10
    x = waste_memory.run()
```